

Phụ lục A — Python cho chuỗi thời gian

Bản tra cứu các hàm NumPy và pandas hay dùng với chuỗi thời gian. Mỗi ví dụ in **output thật** dưới dạng chú thích `#`. Buổi 3 dạy đầy đủ kèm lab. Cần biết Python cơ bản; không cần biết trước pandas.

Code chạy ngày 2026-09-18 trong môi trường buổi 3; các dòng ghi "pandas 2.3.3" chạy trong môi trường buổi 14. Phiên bản:

Python	pandas	NumPy	polars	DuckDB
3.12	3.0.5	2.5.3	1.44.2	1.5.5

Mỗi ví dụ giả định đã có:

```
import numpy as np
import pandas as pd
```

Vì sao có hai bản pandas trong khoá? Tới tháng 9/2026, statsforecast, utilsforecast, mlforecast và autogluon.timeseries vẫn chưa chạy với pandas 3 (ra đầu năm 2026). Buổi nào cần chúng thì dùng pandas 2.3.3 trong môi trường riêng. Chỗ khác nhau: mục 9.

1. Môi trường của một buổi

Môi trường là bộ thư viện đúng phiên bản mà buổi đó cần. Đứng ở `lab/` của buổi:

```
python lab.py up                # dựng môi trường + tải dữ liệu (uv; dùng conda thì thêm --
pip)
python lab.py notebook          # mở JupyterLab ở code/ – notebook của Lab là code/lab.ipynb
python lab.py chạy ../code/vi_du.py # chạy một tệp .py bằng Python của buổi
```

VS Code cũng được: mở `code/lab.ipynb`, chọn Python `lab/00-nen/.venv` (hoặc môi trường conda của bạn).

Gói `tv`. Mỗi môi trường có gói trợ giúp `tv` của khoá (`import tv`). Ví dụ `tv.THU_MUC_DU_LIEU` là đường dẫn tới thư mục dữ liệu đã kiểm tra.

2. NumPy tối thiểu

Mảng. NumPy làm việc với **mảng** (array): một dãy số cùng kiểu. Phép tính trên mảng áp dụng cho **từng phần tử**, không cần vòng `for`.

```
a = np.array([7, 5, 9, 6, 8])
a * 2          # array([14, 10, 18, 12, 16])  – nhân từng số với 2
a + 1          # array([ 8,  6, 10,  7,  9])
a > 6          # array([ True, False,  True, False,  True])
a[a > 6]       # array([7, 9, 8])              – chỉ giữ các số lớn hơn 6
a.mean()      # 7.0                          – trung bình
```

`a[a > 6]` gọi là **lọc bằng điều kiện**: `a > 6` cho một mảng True/False; đặt nó trong ngoặc vuông thì chỉ giữ các vị trí True.

Giá trị thiếu ghi là `np.nan` ("not a number"). Trong **NumPy**, phép tính có `nan` ra `nan`, trừ các hàm bỏ qua nó như `np.nanmean`. Trong **pandas** (`pd.Series` là một cột số, mục 3), `.sum()` và `.mean()` **mặc định bỏ qua** `nan`:

```
b = np.array([7.0, np.nan, 9.0])
b.mean()          # nan    - một ô thiếu làm hỏng cả trung bình
np.nanmean(b)     # 8.0    - bỏ ô thiếu: (7 + 9) / 2
pd.Series(b).mean()          # 8.0    - pandas tự bỏ ô thiếu
pd.Series(b).mean(skipna=False) # nan    - muốn giống NumPy thì tắt bỏ qua
pd.Series([np.nan, np.nan]).sum() # 0.0    - toàn thiếu mà ra 0, xem mục 4
```

Số ngẫu nhiên có seed. Seed (hạt giống) là con số khởi động bộ sinh số ngẫu nhiên: cùng seed thì ra cùng dãy số, nên chạy lại được. Dùng bộ sinh riêng `np.random.default_rng(seed)`, đừng dùng `np.random.seed(...)`: cách cũ này đặt seed chung cho cả chương trình, dễ bị code khác làm lệch.

```
rng = np.random.default_rng(42)
rng.normal(size=3).round(4)      # array([ 0.3047, -1.04 ,  0.7505]) - 3 số từ phân phối chuẩn
rng = np.random.default_rng(42)  # tạo lại với cùng seed...
rng.normal(size=3).round(4)      # array([ 0.3047, -1.04 ,  0.7505]) - ...ra đúng 3 số cũ
```

Quantile. `np.quantile(a, q)` trả về quantile mức `q`, với `q` từ 0 tới 1 (0,25 là 25%). Quantile mức `q` là mốc mà khoảng `q` phần số liệu nằm dưới hoặc bằng nó. **Cách đếm tay:** xếp tăng dần, lấy số đứng ở vị trí $q \times n$ làm tròn lên (n là số giá trị). Với `[1, 2, 3, 4]` và `q = 0.25`: $0,25 \times 4 = 1$, lấy số thứ 1, được **1**.

Mặc định `np.quantile` không đếm như vậy mà **nội suy** giữa hai số kế nhau, nên có thể ra số không có trong dữ liệu. Có 9 cách tính khác nhau (Hyndman & Fan, 1996), chọn bằng tham số `method`:

```
np.quantile([1, 2, 3, 4], 0.25)          # 1.75    mặc định (method="linear")
np.quantile([1, 2, 3, 4], 0.25, method="inverted_cdf") # 1        khớp cách đếm tay
```

Nội suy là lấy một điểm nằm **giữa** hai số kế nhau, theo tỷ lệ. Cách mặc định:

1. Xếp tăng dần, đánh số vị trí **từ 0**: các số 1, 2, 3, 4 ở vị trí 0, 1, 2, 3.
2. Vị trí cần lấy = $(n - 1) \times q$, với n là số giá trị. Ở đây $(4 - 1) \times 0,25 = 0,75$.
3. Vị trí 0,75 nằm giữa vị trí 0 (số 1) và vị trí 1 (số 2), nên kết quả = $1 + 0,75 \times (2 - 1) = 1,75$.

So kết quả với R (một ngôn ngữ thống kê) hay thư viện khác thì kiểm `method` trước khi nghi code sai.

3. pandas tối thiểu và mốc thời gian

Series, DataFrame và chỉ mục

- **DataFrame**: một bảng gồm nhiều cột có tên, giống một trang tính Excel.
- **Series**: một **cột** của bảng, lấy ra bằng một cặp ngoặc vuông: `df["kwh"]`.

Mỗi dòng có một **nhãn dòng**, gọi là **chỉ mục** (index), in ở cột ngoài cùng bên trái; mặc định là 0, 1, 2...
Mỗi cột có một **kiểu** (dtype): `int64` là số nguyên, `float64` là số thực (có phần thập phân), `str` là chữ.

```
df = pd.DataFrame({"tinh": ["HN", "HN", "HCM"], "kwh": [10, 12, 20]})
#   tinh kwh
# 0   HN   10
# 1   HN   12
# 2   HCM   20
df.dtypes                # tinh -> str, kwh -> int64
df["kwh"].tolist()       # [10, 12, 20] - df["kwh"] là một Series
df.loc[1, "kwh"]         # 12 - .loc[nhãn dòng, tên cột]
df.loc[df["kwh"] > 11, "tinh"].tolist() # ['HN', 'HCM'] - .loc[điều kiện dòng, tên cột]
```

`.loc[dòng, cột]` chọn theo **nhãn**: phần trước dấu phẩy chọn dòng (một nhãn, hoặc một dãy True/False), phần sau chọn cột. Cách hai lần ngoặc vuông `df["kwh"][df["kwh"] > 11]` chọn được, nhưng không được dùng để **gán** (mục 9).

Cột số nguyên có ô thiếu sẽ thành số thực. `nan` là một số thực, nên pandas đổi cả cột sang `float64`.

```
pd.Series([1, 2]).dtype      # int64
pd.Series([1, None]).tolist() # [1.0, nan] - kiểu float64: 1 thành 1.0
```

groupby — gộp theo nhóm. `df.groupby("tinh")["kwh"].sum()` chia bảng thành từng nhóm theo giá trị cột `tinh`, rồi cộng cột `kwh` trong mỗi nhóm:

```
df.groupby("tinh")["kwh"].sum()
# tinh
# HCM    20
# HN     22    ← 10 + 12
```

`resample` (mục 4) là một kiểu `groupby` nhóm theo khoảng thời gian thay vì theo giá trị một cột.

Timestamp và `pd.to_datetime`

Một **mốc thời gian** trong pandas là một `Timestamp`, biết cả ngày, giờ, thứ:

```
t = pd.Timestamp("2024-03-10 19:30")
t.hour, t.dayofweek, t.day_name() # (19, 6, 'Sunday') - thứ Hai là 0, Chủ nhật là 6
```

Dữ liệu đọc từ CSV thường là **chuỗi chữ** như `"2024-01-05 08:00"`. `pd.to_datetime` đổi chuỗi thành mốc thời gian; sau đó `.dt` lấy từng phần:

```
s = pd.to_datetime(pd.Series(["2024-01-05 08:00", "2024-01-06 09:30"]))
s.dt.hour.tolist()      # [8, 9]
s.dt.dayofweek.tolist() # [4, 5] - thứ Sáu, thứ Bảy
```

Ba tham số hay cần:

- `format`: mẫu của chuỗi, ví dụ `format="%Y-%m-%d %H:%M"` (năm-tháng-ngày giờ:phút), để khỏi đoán nhầm.
- `dayfirst=True`: chuỗi viết ngày trước tháng, kiểu Việt Nam. Thiếu nó, `"05/01/2024"` bị hiểu là **1 tháng 5**:

```
pd.to_datetime("05/01/2024") # 2024-05-01 – hiểu là tháng 5, ngày 1
pd.to_datetime("05/01/2024", dayfirst=True) # 2024-01-05 – ngày 5, tháng 1
```

- `errors="coerce"`: chuỗi nào không đọc được thì thành `NaT` (giá trị thời gian bị thiếu, giống `nan`), thay vì báo lỗi dừng chương trình:

```
pd.to_datetime(pd.Series(["2024-01-05", "không rõ"]))
# ValueError: time data "không rõ" doesn't match format...
pd.to_datetime(pd.Series(["2024-01-05", "không rõ"]), errors="coerce").tolist()
# [Timestamp('2024-01-05 00:00:00'), NaT]
```

Dùng `errors="coerce"` xong thì luôn đếm số `NaT` bằng `s.isna().sum()` (`.isna()` cho True ở ô thiếu, `.sum()` đếm số True), để không mất dữ liệu âm thầm.

pd.date_range và bí danh tần suất

`pd.date_range(bat_dau, periods=n, freq=...)` tạo `n` mốc thời gian cách đều nhau. **Tần suất** (khoảng cách giữa hai mốc liên nhau) viết bằng một **bí danh** ngắn:

Bí danh	Nghĩa	Ví dụ 3 mốc đầu từ 1/1/2024
<code>s, min, h</code>	giây, phút, giờ	<code>h</code> : 00:00, 01:00, 02:00
<code>15min</code>	15 phút (số đứng trước là bội số)	00:00, 00:15, 00:30
<code>D</code>	ngày	1/1, 2/1, 3/1
<code>W-MON</code>	tuần, mốc là thứ Hai	1/1, 8/1, 15/1
<code>W</code>	tuần, mốc là Chủ nhật (giống <code>W-SUN</code>)	7/1, 14/1, 21/1
<code>MS</code>	tháng, mốc là ngày đầu tháng	1/1, 1/2, 1/3
<code>ME</code>	tháng, mốc là ngày cuối tháng	31/1, 29/2, 31/3
<code>QE, YE</code>	quý, năm, mốc là ngày cuối kỳ	31/3, 30/6, 30/9 (quý)

```
pd.date_range("2024-01-01", periods=3, freq="MS")
# DatetimeIndex(['2024-01-01', '2024-02-01', '2024-03-01'], dtype='datetime64[us]', freq='MS')
pd.date_range("2024-01-01", periods=3, freq="ME")
# DatetimeIndex(['2024-01-31', '2024-02-29', '2024-03-31'], dtype='datetime64[us]', freq='ME')
```

`dtype='datetime64[us]'`: pandas lưu mỗi mốc bằng số **microsecond** (một phần triệu giây), xem mục 9.

Series có chỉ mục thời gian

Chuỗi thời gian trong pandas thường là một `Series` có chỉ mục là các mốc thời gian. Khi đó cắt theo ngày tháng bằng `.loc`:

```
y = pd.Series([5, 7, 6, 9, 8, 10], index=pd.date_range("2024-01-01", periods=6, freq="D"))
y.loc["2024-01-03":"2024-01-04"] # 2024-01-03 → 6, 2024-01-04 → 9 (lấy CẢ hai đầu)
y.loc["2024-01"].sum() # 45 – cộng mọi ngày của tháng 1/2024
```

Khác list Python (`a[2:4]` bỏ vị trí 4), `.loc` với mốc thời gian lấy **cả** mốc cuối.

4. resample — đổi tần suất

Nó làm gì. `resample` gộp dữ liệu theo khoảng thời gian dài hơn, ví dụ từ giờ lên ngày, rồi phải nói cách gộp: `.sum()`, `.mean()`, `.last()` (giá trị cuối)...

Cộng hay trung bình? Tùy loại đại lượng:

- **Lượng tích lũy** (kWh, số chuyến xe, số đơn hàng): **cộng**.
- **Trạng thái tại một lúc** (nhiệt độ, giá, số người đang online): **trung bình** hoặc giá trị cuối. Cộng nhiệt độ 24 giờ không có nghĩa gì.

```
y = pd.Series(range(6), index=pd.date_range("2024-01-01 00:00", periods=6, freq="h")) # 0, 1, ..., 5
lúc 00:00...05:00
y.resample("2h").sum()
# 2024-01-01 00:00:00    1    ← 0 + 1  (giờ 00 và 01)
# 2024-01-01 02:00:00    5    ← 2 + 3
# 2024-01-01 04:00:00    9    ← 4 + 5

nhiệt = pd.Series([20.0, 22, 24, 26], index=pd.date_range("2024-01-01", periods=4, freq="h"))
nhiệt.resample("2h").mean().tolist() # [21.0, 25.0] — nhiệt độ lấy trung bình
```

Nhãn và biên của khoảng: label và closed. Giá trị gắn nhãn 02:00 là tổng của 00:00–02:00 hay 02:00–04:00? Hai tham số quyết định:

- **closed**: khoảng **gồm** mốc đầu (`"left"`, khoảng `[02:00, 04:00)`, gồm 02:00, bỏ 04:00) hay gồm mốc cuối (`"right"`, khoảng `(02:00, 04:00]`, bỏ 02:00, gồm 04:00).
- **label**: nhãn ghi bằng mốc đầu (`"left"`) hay mốc cuối (`"right"`) của khoảng.

Mặc định cả hai là `"left"`, **trừ** các tần suất gắn với cuối kỳ (`ME`, `QE`, `YE`, `W...`) mặc định `"right"`. Cùng dữ liệu trên, đổi sang `"right"`:

```
y.resample("2h", closed="right", label="right").sum()
# 2024-01-01 00:00:00    0    ← khoảng (22:00 hôm trước, 00:00]: chỉ có giờ 00
# 2024-01-01 02:00:00    3    ← (00:00, 02:00]: giờ 01 + giờ 02 = 1 + 2
# 2024-01-01 04:00:00    7    ← 3 + 4
# 2024-01-01 06:00:00    5    ← chỉ có giờ 05
```

Dữ liệu điện hay ghi "giờ kết thúc" (nhãn 01:00 là điện dùng từ 00:00 tới 01:00); khi đó `"right"` mới đúng nghĩa. Đọc tài liệu nguồn dữ liệu trước khi chọn.

ME và MS. Cùng gộp theo tháng, chỉ khác nhãn:

```
d = pd.Series([1.0, 2, 3, 4],
               index=pd.to_datetime(["2024-01-30", "2024-01-31", "2024-02-01", "2024-02-02"]))
d.resample("ME").sum() # 2024-01-31 → 3.0, 2024-02-29 → 7.0
d.resample("MS").sum() # 2024-01-01 → 3.0, 2024-02-01 → 7.0
```

min_count — đừng để khoảng trống thành số 0. `.sum()` của khoảng không có số liệu ra **0**, mà "0 kWh" khác "không đo được". `min_count=k`: khoảng nào có ít hơn `k` giá trị thật thì để trống (`nan`):

```
z = pd.Series([1.0, np.nan, np.nan, np.nan], index=pd.date_range("2024-01-01", periods=4, freq="h"))
z.resample("2h").sum().tolist()           # [1.0, 0.0] - khoảng thứ hai toàn nan mà ra 0
z.resample("2h").sum(min_count=1).tolist() # [1.0, nan] - đúng: không có số liệu
```

Tự kiểm tra. Chuỗi 1, 2, 3, 4 lúc 00:00, 01:00, 02:00, 03:00. Đoán kết quả của `resample("2h").sum()` và `resample("2h", closed="right", label="right").sum()`.

Đáp án (*tự làm trước rồi mới đọc*)

Mặc định ("`left`"): [00:00, 02:00) gồm giờ 00 và 01, được $1 + 2 = 3$; [02:00, 04:00) được $3 + 4 = 7$. Kết quả 3, 7.

Với "`right`": (22:00, 00:00] chỉ có giờ 00, được 1; (00:00, 02:00] gồm giờ 01 và 02, được $2 + 3 = 5$; (02:00, 04:00] chỉ có giờ 03, được 4. Kết quả 1, 5, 4 với nhãn 00:00, 02:00, 04:00. Nhầm hay gặp là giữ nguyên số khoảng, chỉ đổi nhãn.

5. Dời và cửa sổ: shift, diff, rolling, ewm

Các hàm này tạo **feature** (đặc trưng đầu vào cho mô hình) từ chính chuỗi. Với mỗi hàm, hỏi: **tại thời điểm t , nó dùng dữ liệu của những thời điểm nào?** Dùng dữ liệu **sau** lúc ra dự báo là **rò rỉ tương lai**: mô hình "nhìn trộm" tương lai, điểm đánh giá đẹp giả tạo rồi hỏng khi dùng thật. Ví dụ: cuối giờ 10 dự báo giờ 11 thì đã biết y_{10} , dùng được; nhưng nếu dùng t là để đoán chính y_t thì mọi feature ở dòng đó chỉ được dùng tới y_{t-1} .

Chuỗi dùng chung, 6 giờ:

```
x = pd.Series([0.0, 1, 2, 3, 4, 5], index=pd.date_range("2024-01-01", periods=6, freq="h"))
```

shift — dời chuỗi

`shift(1)` đẩy mọi giá trị xuống 1 bước: tại giờ t ta thấy giá trị của giờ $t - 1$ (gọi là **trễ 1**); ô đầu không có giờ trước nên thành `nan`. `shift(-1)` đẩy ngược lên, tức lấy giá trị của **giờ sau**, là tương lai.

```
x.shift(1).tolist()      # [nan, 0.0, 1.0, 2.0, 3.0, 4.0] - trễ 1: an toàn
x.shift(-1).tolist()     # [1.0, 2.0, 3.0, 4.0, 5.0, nan] - giờ sau: KHÔNG được dùng làm feature
```

diff — chênh lệch với bước trước

`diff()` tính $y_t - y_{t-1}$: giá trị hiện tại trừ giá trị ngay trước.

```
x.diff().tolist()       # [nan, 1.0, 1.0, 1.0, 1.0, 1.0]
```

Nó dùng y_t , nên chỉ làm feature được nếu lúc ra dự báo đã biết y_t . Nếu y_t chính là thứ cần dự báo, dùng `x.diff().shift(1)`.

rolling — cửa sổ trượt

`rolling(3).mean()` tính trung bình của **3 giá trị gần nhất, tính cả giá trị hiện tại**. Hai ô đầu chưa đủ 3 giá trị nên là `nan`.

```
x.rolling(3).mean().tolist() # [nan, nan, 1.0, 2.0, 3.0, 4.0]
```

Tính tay ô thứ 3 (giờ 02:00): $(0 + 1 + 2) / 3 = 1,0$.

- `min_periods=1`: tính luôn khi chưa đủ 3 giá trị.
- `center=True`: đặt cửa sổ **giữa** t , tức dùng y_{t-1} , y_t , y_{t+1} ; y_{t+1} là tương lai.
- Muốn cửa sổ chỉ gồm quá khứ trước t : thêm `.shift(1)`.

```
x.rolling(3, min_periods=1).mean().tolist() # [0.0, 0.5, 1.0, 2.0, 3.0, 4.0]
x.rolling(3, center=True).mean().tolist()   # [nan, 1.0, 2.0, 3.0, 4.0, nan]
                                             # ô giờ 01 dùng cả giờ 02
x.rolling(3).mean().shift(1).tolist()       # [nan, nan, nan, 1.0, 2.0, 3.0]
                                             # chỉ dùng 3 giờ TRƯỚC t
```

Cửa sổ theo khoảng thời gian. `rolling("2h")` lấy "mọi giá trị trong 2 giờ vừa qua", còn `rolling(2)` lấy "2 giá trị gần nhất". Hai cách khác nhau khi dữ liệu có chỗ trống:

```
g = pd.Series([1.0, 2, 3],
               index=pd.to_datetime(["2024-01-01 00:00", "2024-01-01 01:00", "2024-01-01 05:00"]))
g.rolling(2).sum().tolist() # [nan, 3.0, 5.0]
                           # ô 05:00 cộng cả giá trị lúc 01:00, cách 4 giờ
g.rolling("2h").sum().tolist() # [1.0, 3.0, 3.0]
                              # ô 05:00 chỉ còn chính nó, vì 01:00 đã quá 2 giờ
```

Ô đầu của `rolling("2h")` có số vì cửa sổ theo **thời gian** mặc định `min_periods=1`; cửa sổ theo **số giá trị** đòi đủ số giá trị.

Với cửa sổ thời gian, `closed` quyết định hai đầu cửa sổ có được tính không. Mặc định `"right"`: khoảng $(t - 2h, t]$, bỏ mốc cách đúng 2 giờ, gồm t .

```
x.rolling("2h").sum().tolist() # [0.0, 1.0, 3.0, 5.0, 7.0, 9.0] (t-2h, t]
x.rolling("2h", closed="both").sum().tolist() # [0.0, 1.0, 3.0, 6.0, 9.0, 12.0] [t-2h, t]
x.rolling("2h", closed="left").sum().tolist() # [nan, 0.0, 1.0, 3.0, 5.0, 7.0] [t-2h, t)
                                             # không gồm t
```

Tính tay ô giờ 03:00 với `"both"`: gồm giờ 01, 02, 03, tổng $1 + 2 + 3 = 6$. Với mặc định bỏ giờ 01: $2 + 3 = 5$.

Ô đầu với `closed="left"` là `nan` vì cửa sổ [22:00, 00:00) rỗng, chưa đủ `min_periods=1`.

ewm — trung bình trượt hàm mũ

`ewm(alpha=...).mean()` là trung bình mà **giá trị càng mới càng nặng ký**. Dạng đệ quy (`adjust=False`):

$$s_t = \alpha y_t + (1 - \alpha) s_{t-1}$$

- y_t : giá trị mới tại t ; s_t : giá trị đã làm trơn tại t ; s_{t-1} : giá trị đã làm trơn ở bước trước.
- α (alpha): trọng số của giá trị mới, từ 0 tới 1. α lớn thì bám sát giá trị mới; nhỏ thì trơn hơn.

Nói bằng lời. Giá trị trơn mới = α phần giá trị mới + $(1 - \alpha)$ phần giá trị trơn cũ. Với chuỗi 1, 2, 3 và $\alpha = 0,5$: $s_1 = 1$; $s_2 = 0,5 \times 2 + 0,5 \times 1 = 1,5$; $s_3 = 0,5 \times 3 + 0,5 \times 1,5 = 2,25$.

```
pd.Series([1.0, 2, 3]).ewm(alpha=0.5, adjust=False).mean().tolist() # [1.0, 1.5, 2.25]
pd.Series([1.0, 2, 3]).ewm(alpha=0.5).mean().round(3).tolist() # [1.0, 1.667, 2.429]
```

Mặc định `adjust=True` cho số khác ở các bước đầu: nó lấy trung bình có trọng số (mỗi số nhân trọng số của nó, cộng lại, chia tổng trọng số) của mọi giá trị đã có, trọng số 1, $(1 - \alpha)$, $(1 - \alpha)^2$... từ mới tới cũ. Tính tay bước 3: trọng số 1; 0,5; 0,25 cho 3, 2, 1, nên $(1 \times 3 + 0,5 \times 2 + 0,25 \times 1) / (1 + 0,5 + 0,25) = 4,25 / 1,75 \approx 2,429$. Càng về sau, hai cách càng gần nhau.

Bảng tổng hợp: hàm nào nhìn trộm tương lai?

Hàm	Tại thời điểm t dùng	Làm feature được không
<code>shift(1)</code>	y_{t-1}	được
<code>shift(-1)</code>	y_{t+1}	không — tương lai
<code>diff()</code>	y_t và y_{t-1}	chỉ khi y_t đã biết lúc ra dự báo
<code>rolling(3).mean()</code>	y_{t-2} , y_{t-1} , y_t	thêm <code>.shift(1)</code> nếu y_t là thứ cần dự báo
<code>rolling(3, center=True).mean()</code>	y_{t-1} , y_t , y_{t+1}	không — có tương lai
<code>ewm(alpha=...).mean()</code>	mọi giá trị tới t	thêm <code>.shift(1)</code> nếu y_t là thứ cần dự báo

Đọc bảng. Cột 2 có y_{t+1} thì cấm làm feature. Có y_t thì hỏi lúc ra dự báo đã biết y_t chưa; nếu y_t là thứ cần dự báo thì thêm `.shift(1)`.

Tự kiểm tra. Chuỗi 10, 20, 30, 40 theo giờ liên tiếp. Ở giờ thứ 4, `rolling(2).mean().shift(1)` ra bao nhiêu? Có dùng giá trị 40 không?

Đáp án (tự làm trước rồi mới đọc)

`rolling(2).mean()` ở giờ thứ 3 là $(20 + 30) / 2 = 25$. `shift(1)` dời nó xuống giờ thứ 4, nên ra **25**, không dùng 40. Nhầm hay gặp là quên `shift(1)`: khi đó giờ thứ 4 ra $(30 + 40) / 2 = 35$, có dùng chính giá trị 40 đang cần dự báo.

6. merge_asof — ghép với giá trị gần nhất trong quá khứ

Nó làm gì. Ghép hai bảng theo thời gian khi mốc **không trùng nhau**: mỗi dòng bảng trái lấy dòng bảng phải **gần nhất, trước hoặc đúng bằng** nó. Đơn hàng lúc 10:07 phải lấy giá cập nhật lúc 10:05, không lấy giá 10:10, vì lúc 10:07 giá đó chưa có (lấy là **rò rỉ tương lai**, mục 5).

```
don = pd.DataFrame({"t": pd.to_datetime(["2024-01-01 10:07"])})
gia = pd.DataFrame({"t": pd.to_datetime(["2024-01-01 10:05", "2024-01-01 10:10"]), "gia": [100, 105]})
pd.merge_asof(don, gia, on="t") # t = 10:07, gia = 100 — lấy giá lúc 10:05
pd.merge(don, gia, on="t", how="left") # t = 10:07, gia = NaN
# ghép thường đòi trùng khít từng giây
```

`pd.merge` là phép **ghép thường**: chỉ ghép dòng có **t bằng hệt nhau**. `how="left"` giữ mọi dòng bảng trái, dòng không có cặp thì để trống.

Các tham số:

- `on="t"`: cột thời gian để ghép. **Hai bảng phải xếp tăng dần** theo cột này, nếu không sẽ báo `ValueError: right keys must be sorted`.
- `direction="backward"` (mặc định): chỉ nhìn về quá khứ. `"forward"` nhìn về tương lai, dễ gây rò rỉ.
- `allow_exact_matches=False`: không lấy dòng có thời điểm **trùng đúng**, chỉ lấy dòng trước đó. Dùng khi giá trị tại đúng thời điểm `t` chưa kịp công bố lúc ra dự báo.
- `tolerance=pd.Timedelta("1min")`: chỉ ghép nếu dòng quá khứ cách không quá 1 phút.

```
tra = pd.DataFrame({"t": pd.to_datetime(["2024-01-01 10:00", "2024-01-01 10:05"])})
phai = pd.DataFrame({"t": pd.to_datetime(["2024-01-01 10:00", "2024-01-01 10:03"]), "v": [1, 2]})
pd.merge_asof(tra, phai, on="t")["v"].tolist() # [1, 2]
pd.merge_asof(tra, phai, on="t", allow_exact_matches=False)["v"].tolist() # [nan, 2.0]
pd.merge_asof(tra, phai, on="t", tolerance=pd.Timedelta("1min"))["v"].tolist() # [1.0, nan]
```

Lệnh thứ 2: dòng 10:00 bị cấm lấy dòng trùng 10:00 bên phải, trước đó không còn gì, nên `nan`. Lệnh thứ 3: dòng 10:05 cách 10:03 hai phút, quá giới hạn 1 phút, nên `nan`. Cột có `nan` nên 1 thành 1.0 (mục 3).

Tự kiểm tra. Bảng trái có một dòng lúc 10:02. Bảng phải có 10:00 ($v = 1$), 10:02 ($v = 2$), 10:04 ($v = 3$). `merge_asof` mặc định cho `v` bằng mấy? Thêm `allow_exact_matches=False` thì sao?

Đáp án (tự làm trước rồi mới đọc)

Mặc định lấy dòng gần nhất **trước hoặc đúng bằng** 10:02, tức 10:02, nên $v = 2$. Cấm lấy trùng thì lùi về 10:00, $v = 1$. Không bao giờ lấy 3, vì 10:04 nằm sau 10:02 (trừ khi đặt `direction="forward"`).

7. Múi giờ và giờ mùa hè

Quy tắc của khoá. Lưu mọi mốc thời gian theo **UTC** (giờ chuẩn quốc tế, không đổi theo mùa). Chỉ đổi sang giờ địa phương khi hiển thị, hoặc khi tính feature lịch như "giờ trong ngày", "thứ trong tuần".

Độ lệch so với UTC viết sau giờ: `+07:00` là giờ địa phương **nhANH hơn** UTC 7 giờ (7:00 sáng ở Hà Nội là 0:00 UTC); `-05:00` là **chẬM hơn** UTC 5 giờ (New York mùa đông).

Vì sao phải thống nhất múi giờ. Bảng taxi New York ghi giờ địa phương, bảng mưa ghi UTC, cả hai không ghi múi giờ. Mưa lúc 21:00 UTC, tức 17:00 New York (mùa hè), đúng lúc chuyển xe tăng vọt. Ghép thẳng theo con số giờ thì mưa hiện sau đợt tăng 4 giờ, và ta kết luận sai "xe tăng **trước khi** mưa".

Naive và aware

- Mốc **naive** không ghi múi giờ: `2024-07-01 12:00`. Không biết đó là 12 giờ ở đâu.
- Mốc **aware** có ghi múi giờ: `2024-07-01 12:00+07:00`.
- `tz_localize(múi_giờ)`: **gắN** múi giờ cho mốc naive, **giữ nguyên** con số giờ.
- `tz_convert(múi_giờ)`: **đổI** một mốc aware sang múi giờ khác. Thời điểm thật không đổi, con số giờ đổi.

```
t = pd.Timestamp("2024-07-01 12:00") # naive
ta = t.tz_localize("Asia/Ho_Chi_Minh") # 2024-07-01 12:00:00+07:00
# "12 giờ ở Việt Nam"
ta.tz_convert("UTC") # 2024-07-01 05:00:00+00:00
```

```

t.tz_localize("UTC")
# cùng lúc đó, ở UTC là 5 giờ
# 2024-07-01 12:00:00+00:00
# "12 giờ UTC", một thời điểm KHÁC

t < ta
# TypeError: Cannot compare tz-naive and tz-aware timestamps

```

Tên múi giờ lấy từ **cơ sở dữ liệu múi giờ IANA** (danh sách chuẩn quốc tế ghi mọi lần một nơi đổi giờ trong lịch sử), dạng `Châu_lục/Thành_phố`: `Asia/Ho_Chi_Minh`, `America/New_York`, `Europe/London`.

Feature lịch phải tính theo giờ địa phương. 17:00 UTC ngày 1/1 là 0:00 ngày 2/1 ở Việt Nam:

```

u = pd.Series(pd.to_datetime(["2024-01-01 17:00", "2024-01-01 18:00"])).dt.tz_localize("UTC")
u.dt.hour.tolist()
# [17, 18] - giờ UTC
u.dt.tz_convert("Asia/Ho_Chi_Minh").dt.hour.tolist()
# [0, 1] - giờ Việt Nam, đã sang ngày 2/1

```

Việt Nam không phải lúc nào cũng UTC+7. Theo IANA, Việt Nam chỉ cố định ở UTC+7 từ 13/6/1975; trước đó có giai đoạn lịch khác:

```

from datetime import datetime
import zoneinfo
zoneinfo.ZoneInfo("Asia/Ho_Chi_Minh").utcoffset(datetime(1970, 1, 1))
# 8:00:00
# năm 1970 là UTC+8
zoneinfo.ZoneInfo("Asia/Ho_Chi_Minh").utcoffset(datetime(2024, 1, 1))
# 7:00:00

```

Vì vậy đừng tự cộng 7 giờ; hãy dùng `tz_localize/tz_convert` với tên múi giờ.

Giờ mùa hè (DST)

Một số nước (Mỹ, châu Âu) **vận động hồ nhanh 1 giờ** vào mùa xuân và vận lại vào mùa thu, gọi là giờ mùa hè (DST, daylight saving time). Hệ quả với dữ liệu theo giờ:

- Ngày vận nhanh có **23 giờ**: một giờ **bị mất**. Ở New York ngày 10/3/2024 không có 02:00-02:59.
- Ngày vận lại có **25 giờ**: một giờ **bị lặp**. Ngày 3/11/2024 có hai lần 01:00-01:59.

```

pd.date_range("2024-03-10 00:00", periods=4, freq="h", tz="America/New_York")
# 00:00-05:00, 01:00-05:00, 03:00-04:00, 04:00-04:00 ← không có 02:00 ("-05:00" là độ lệch UTC,
không phải giờ kết thúc)
pd.date_range("2024-11-03 00:00", periods=4, freq="h", tz="America/New_York")
# 00:00-04:00, 01:00-04:00, 01:00-05:00, 02:00-05:00 ← 01:00 xuất hiện hai lần

```

Mùa đông New York là -05:00; vận nhanh 1 giờ thành -04:00. Dữ liệu điện EIA (Cơ quan Thông tin Năng lượng Mỹ) và taxi New York gặp hai ngày này mỗi năm.

Gắn múi giờ cho giờ bị mất hoặc bị lặp sẽ báo lỗi, trừ khi nói rõ cách xử lý:

```

t = pd.Series(pd.to_datetime(["2024-11-03 01:30"]))
t.dt.tz_localize("America/New_York")
# ValueError: Cannot infer dst time from 2024-11-03 01:30:00, try using the 'ambiguous' argument
t.dt.tz_localize("America/New_York", ambiguous=np.array([True]))
# 01:30-04:00
# lần thứ nhất (còn giờ mùa hè)
t.dt.tz_localize("America/New_York", ambiguous=np.array([False]))
# 01:30-05:00 - lần thứ hai

m = pd.Series(pd.to_datetime(["2024-03-10 02:30"]))
# giờ không tồn tại

```

```
m.dt.tz_localize("America/New_York")
# ValueError: 2024-03-10 02:30:00 is a nonexistent time due to daylight savings time.
# Try using the 'nonexistent' argument.
m.dt.tz_localize("America/New_York", nonexistent="shift_forward") # 03:00-04:00
# đẩy tới giờ có thật kế tiếp
```

Các lựa chọn:

- **ambiguous** (giờ bị lặp): **"infer"** tự suy từ thứ tự các dòng; **"NaT"** để trống; **"raise"** báo lỗi (mặc định); hoặc một mảng True/False, True là lần thứ nhất (còn giờ mùa hè). Phải là **mảng**, mỗi dòng một True/False.
- **nonexistent** (giờ bị mất): **"shift_forward"** đẩy tới giờ kế tiếp; **"shift_backward"** lùi về giờ trước; **"NaT"** để trống; một khoảng thời gian (ví dụ `pd.Timedelta("1h")`) để cộng thêm; **"raise"** báo lỗi (mặc định).

"infer" cần các dòng xếp đúng thứ tự, và giờ lặp phải xuất hiện đủ hai lần:

```
seq = pd.Series(pd.to_datetime(["2024-11-03 00:30", "2024-11-03 01:30",
                                "2024-11-03 01:30", "2024-11-03 02:30"]))
seq.dt.tz_localize("America/New_York", ambiguous="infer").tolist()
# [00:30-04:00, 01:30-04:00, 01:30-05:00, 02:30-05:00] – hai lần 01:30 được tách đúng
```

Tự kiểm tra. Một cảm biến ghi `2024-07-01 20:00` theo UTC. Ở Việt Nam đó là mấy giờ, ngày nào? Nếu tính feature "ngày trong tuần" bằng giờ UTC thì sai ở đâu?

Đáp án (tự làm trước rồi mới đọc)

Đổi sang `Asia/Ho_Chi_Minh` (năm 2024 là UTC+7): **03:00 ngày 2/7**. Tính bằng giờ UTC thì số đo bị xếp vào 20 giờ thứ Hai thay vì 3 giờ sáng thứ Ba: lệch cả giờ lẫn thứ.

8. Định dạng dài và định dạng rộng

Nhiều chuỗi (ví dụ tải điện của nhiều tỉnh) có thể xếp theo hai cách:

- **Dạng rộng:** mỗi chuỗi một cột, mỗi dòng một thời điểm.
- **Dạng dài:** mỗi dòng là **một** quan sát của **một** chuỗi tại **một** thời điểm. Các thư viện Nixtla (`statsforecast`, `mlforecast`, `neuralforecast`) đòi dạng dài với đúng ba cột: `unique_id` (tên chuỗi), `ds` (thời điểm), `y` (giá trị).

`melt` đổi rộng thành dài; `pivot` đổi dài thành rộng:

```
rong = pd.DataFrame({"ds": pd.date_range("2024-01-01", periods=2, freq="D"), "HN": [10, 11], "HCM": [20, 21]})
#           ds  HN  HCM
# 0 2024-01-01   10   20
# 1 2024-01-02   11   21

dai = rong.melt(id_vars="ds", var_name="unique_id", value_name="y")[["unique_id", "ds", "y"]]
#  unique_id      ds  y
# 0         HN 2024-01-01  10
# 1         HN 2024-01-02  11
```

```
# 2      HCM 2024-01-01  20
# 3      HCM 2024-01-02  21
```

```
dai.pivot(index="ds", columns="unique_id", values="y")    # quay lại dạng rộng (cột xếp theo tên: HCM, HN)
```

Trong `melt`: `id_vars` là cột giữ nguyên; tên các cột còn lại (`HN`, `HCM`) vào cột `var_name`, giá trị vào cột `value_name`.

Dạng dài chứa được các chuỗi dài ngắn khác nhau và dễ thêm cột feature. Dạng rộng tiện để vẽ nhiều chuỗi cạnh nhau và tính tương quan giữa các chuỗi.

9. pandas 2 và pandas 3 — những chỗ khác nhau

Chủ đề	pandas 2.3.3	pandas 3.0.5
Gán qua hai lần ngoặc vuông: <code>df["a"][mask] = v</code>	có tác dụng , kèm <code>FutureWarning</code>	không có tác dụng , kèm <code>ChainedAssignmentError</code>
Kiểu của cột chữ	<code>object</code> (kiểu chung "đối tượng Python bất kỳ")	<code>str</code>
<code>pd.to_datetime</code> từ chuỗi	<code>datetime64[ns]</code> (nano giây)	<code>datetime64[us]</code> (micro giây)
Bí danh tần suất cũ <code>H</code> , <code>T</code> , <code>M</code>	chạy, kèm <code>FutureWarning</code>	<code>ValueError: Invalid frequency</code>
<code>fillna(method="ffill")</code>	chạy, kèm <code>FutureWarning</code>	<code>TypeError</code> — dùng <code>.ffill()</code> (điền ô trống bằng giá trị trước nó)
Kiểu đối tượng múi giờ	của gói <code>pytz</code>	của thư viện chuẩn <code>zoneinfo</code> (pandas không cài kèm <code>pytz</code> nữa)
<code>groupby</code> trên cột phân loại có nhóm rỗng	vẫn in nhóm rỗng (<code>observed=False</code>)	bỏ nhóm rỗng (<code>observed=True</code>)

Bí danh cũ và mới (đã chạy thử: bí danh cũ báo `ValueError` trên pandas 3):

Cũ	H	T	S	L	U	M	Q	Y, A	BM
Mới	h	min	s	ms	us	ME	QE	YE	BME
Nghĩa	giờ	phút	giây	mili giây	micro giây	cuối tháng	cuối quý	cuối năm	ngày làm việc cuối tháng

Độc bảng. Nguy hiểm nhất là chỗ **cùng code, kết quả khác mà không dùng**: gán hai lần ngoặc, đơn vị thời gian, `groupby`. Bí danh cũ và `fillna(method=...)` thì báo lỗi ngay trên pandas 3. `FutureWarning` là cảnh báo "cách viết này sẽ bị bỏ ở bản sau".

groupby và nhóm rỗng. Cột **phân loại** (kiểu `category`) là cột chữ có danh sách giá trị khai sẵn. Giá trị chưa xuất hiện dòng nào thành **nhóm rỗng** khi `groupby`: pandas 2 mặc định vẫn in (tổng bằng 0), pandas 3 bỏ:

```
c = pd.DataFrame({"loai": pd.Categorical(["ngay_thuong", "ngay_thuong"],
                                         categories=["ngay_thuong", "ngay_le"]), # khai sẵn 2 giá
                  "kwh": [10, 12]}) # nhưng chỉ có dòng "ngay_thuong"
c.groupby("loai", observed=True)["kwh"].sum() # ngay_thuong → 22 (mặc định pandas)
```

```
3)
c.groupby("loai", observed=False)["kwh"].sum()    # ngay_thuong → 22, ngay_le → 0 (mặc định pandas 2.3.3)
```

Không ghi `observed` thì số nhóm khác nhau giữa hai bản, nên code đếm nhóm hay ghép bảng phía sau lệch âm thầm. Luôn ghi rõ `observed=True` hoặc `observed=False`.

Gán giá trị: luôn dùng `.loc`. pandas 3 bật cơ chế **Copy-on-Write** (chép khi ghi): mọi phần cắt ra từ một bảng cư xử như **bản sao** độc lập, nên gán vào `df["a"]` không đổi `df`.

```
df = pd.DataFrame({"a": [1, 2, 3], "b": [4, 5, 6]})
df["a"][df["b"] > 4] = 100          # pandas 3: ChainedAssignmentError, df KHÔNG đổi
df["a"].tolist()                    # [1, 2, 3] (pandas 2.3.3: [1, 100, 100] kèm FutureWarning)
df.loc[df["b"] > 4, "a"] = 100      # cách đúng: một lần .loc với [điều kiện dòng, tên cột]
df["a"].tolist()                    # [1, 100, 100]
```

Dù tên có chữ "Error", `ChainedAssignmentError` chỉ là **cảnh báo**: chương trình chạy tiếp, `df` không đổi, nên rất dễ bỏ sót.

Đơn vị thời gian. Mỗi mốc thời gian được lưu là số đơn vị kể từ 0:00 UTC ngày 1/1/1970: pandas 2 đếm nano giây (một phần tỷ giây), pandas 3 đếm micro giây. Đổi sang số nguyên thì hai bản lệch 1.000 lần; cần số nguyên thì `.dt.as_unit("ns")` trước:

```
ts = pd.to_datetime(pd.Series(["2024-01-01"]))
ts.astype("int64").tolist()          # pandas 3: [1704067200000000000]
                                     # pandas 2.3.3: [1704067200000000000]
ts.dt.as_unit("ns").astype("int64").tolist() # cả hai bản: [1704067200000000000]
                                     # nói rõ đơn vị thì như nhau
```

10. polars

polars là thư viện bảng dữ liệu khác pandas, chạy nhiều lõi CPU cùng lúc. **Chế độ lazy** (chạy lười): viết cả chuỗi phép tính trước, polars tối ưu rồi mới chạy khi gọi `.collect()`; `pl.scan_parquet(...)` đọc file lớn theo cách này.

```
import polars as pl

lz = pl.LazyFrame({"a": [1, 2, 3]}).filter(pl.col("a") > 1)  # chưa chạy gì
lz.collect()["a"].to_list()                                   # [2, 3] – giờ mới chạy
```

Bốn từ hay gặp:

- `pl.col("v")`: "cột tên `v`", dùng để viết phép tính trên cột đó.
- `.alias("ten")`: đặt tên cho cột kết quả.
- `df.with_columns(...)`: trả về bảng mới có thêm (hoặc thay) các cột tính trong ngoặc.
- `.agg(...)`: sau khi gộp nhóm, nói mỗi nhóm tính gì (tổng, trung bình...).

Gộp theo khoảng thời gian: `group_by_dynamic`, tương đương `resample`. Mặc định `closed="left"`, `label="left"`; cột thời gian phải xếp tăng dần:

```
d = pl.DataFrame({"t": pd.date_range("2024-01-01", periods=6, freq="h"), "v": [0, 1, 2, 3, 4, 5]})
d.groupby_dynamic("t", every="2h").agg(pl.col("v").sum())
# t: 00:00, 02:00, 04:00; v: 1, 5, 9 — giống ví dụ resample ở mục 4
```

Trễ và cửa sổ: `pl.col("v").shift(1)` và `pl.col("v").rolling_mean(3)`. Ô thiếu hiện là `None`. Khác pandas (mục 3), cột số nguyên có ô thiếu vẫn là số nguyên:

```
d.with_columns(pl.col("v").shift(1).alias("tre_1"))["tre_1"].to_list() # [None, 0, 1, 2, 3, 4]
d.with_columns(pl.col("v").rolling_mean(3).alias("tb3"))["tb3"].to_list() # [None, None, 1.0, 2.0, 3.0, 4.0]
```

Ghép quá khứ gần nhất: `join_asof`, tương đương `merge_asof`, mặc định `strategy="backward"`. Hai bảng phải xếp tăng dần theo cột thời gian:

```
q = pl.DataFrame({"t": pd.to_datetime(["2024-01-01 10:00", "2024-01-01 10:05"])}))
r = pl.DataFrame({"t": pd.to_datetime(["2024-01-01 10:00", "2024-01-01 10:03"]), "v": [1, 2]})
q.join_asof(r, on="t")["v"].to_list() # [1, 2] — cùng dữ liệu ví dụ merge_asof ở mục 6
```

Múi giờ — hai hàm dễ nhầm.

- `dt.replace_time_zone(mui_gio)`: gán (hoặc **thay**) nhãn múi giờ, **giữ nguyên con số giờ**, tức đổi luôn thời điểm thật. Giống `tz_localize` của pandas.
- `dt.convert_time_zone(mui_gio)`: giữ nguyên thời điểm thật, đổi con số giờ. Giống `tz_convert`.

```
d.with_columns(pl.col("t").dt.replace_time_zone("UTC")
               .dt.convert_time_zone("Asia/Ho_Chi_Minh"))["t"][0] # 2024-01-01
07:00:00+07:00 # đúng

d.with_columns(pl.col("t").dt.replace_time_zone("UTC")
               .dt.replace_time_zone("Asia/Ho_Chi_Minh"))["t"][0] # 2024-01-01
00:00:00+07:00 # lệch 7 giờ
```

Dòng thứ hai không báo lỗi mà lệch 7 giờ âm thầm.

11. DuckDB

DuckDB là cơ sở dữ liệu chạy ngay trong Python, không cần máy chủ. Truy vấn viết bằng **SQL** (ngôn ngữ truy vấn bảng), chạy thẳng trên file Parquet hay CSV. Parquet lưu bảng theo cột, nén tốt, đọc nhanh hơn CSV.

SQL tối thiểu.

- `SELECT t, y FROM bang_y`: lấy cột `t` và `y` từ bảng `bang_y`. `SELECT *` là lấy mọi cột.
- `WHERE y > 5`: chỉ giữ dòng thoả điều kiện, giống lọc `df[df["y"] > 5]`.
- `ORDER BY t`: xếp dòng theo cột `t`.
- `-- ...`: chú thích, DuckDB bỏ qua phần sau `--` (giống `#` trong Python).
- `TIMESTAMP '2024-01-01 10:07'`: một mốc thời gian viết thẳng trong câu lệnh; `INTERVAL '15 minutes'`: một khoảng thời gian dài 15 phút.

```

SELECT * FROM 'du-lieu.parquet'; -- đọc file trực tiếp

-- gộp theo khoảng thời gian, tương đương resample
SELECT time_bucket(INTERVAL '15 minutes', TIMESTAMP '2024-01-01 10:07'); -- 10:00
SELECT date_trunc('hour', TIMESTAMP '2024-01-01 10:07'); -- 10:00 (cắt về đầu
giờ)

-- trễ 1 bước, tương đương shift(1): cột y = 5, 7, 6 cho tre_1 = NULL, 5, 7
SELECT t, y, lag(y) OVER (ORDER BY t) AS tre_1 FROM bang_y;

-- ghép quá khứ gần nhất, tương đương merge_asof (cùng dữ liệu ví dụ mục 6)
SELECT trai.t, phai.v FROM trai ASOF JOIN phai ON trai.t >= phai.t; -- (10:00, 1), (10:05, 2)

```

ASOF JOIN ... ON trai.t >= phai.t: mỗi dòng **trai** chỉ lấy **một** dòng **phai** gần nhất không sau nó, còn phép ghép thường lấy **mọi** dòng thoả điều kiện. Vì vậy dòng 10:05 lấy 10:03 (v = 2), không lấy 10:00.

Trung bình 3 bước gần nhất, tương đương `rolling(3).mean()`, viết là `avg(y) OVER (ORDER BY t ROWS 2 PRECEDING)`: cửa sổ gồm dòng hiện tại và 2 dòng trước; hai dòng đầu vẫn có số, như `min_periods=1`.

bang_y, trai, phai từ đâu ra? DuckDB tự đọc DataFrame pandas cùng tên đang có trong chương trình như một bảng. **trai**, **phai** ở mục 6; **bang_y** tạo như sau:

```

import duckdb
bang_y = pd.DataFrame({"t": pd.date_range("2024-01-01", periods=3, freq="h"), "y": [5, 7, 6]})
duckdb.sql("SELECT t, y, lag(y) OVER (ORDER BY t) AS tre_1 FROM bang_y ORDER BY t").df()
#           t      y  tre_1
# 0 2024-01-01 00:00:00    5   <NA>
# 1 2024-01-01 01:00:00    7      5
# 2 2024-01-01 02:00:00    6      7

```

Đọc câu SQL:

- `lag(y)`: giá trị `y` của dòng **ngay trước**. `OVER (ORDER BY t)` nói "trước" theo thứ tự nào: xếp theo cột `t`.
- `AS tre_1`: đặt tên cột kết quả, giống `.alias` của polars.
- `.df()`: đổi kết quả về DataFrame pandas. `<NA>` (SQL gọi là `NULL`) là ô thiếu. `tre_1` vẫn là số nguyên vì DuckDB trả kiểu `Int64` (chữ l hoa) của pandas, chứa được ô thiếu, khác `int64` ở mục 3.
- `::VARCHAR` (dùng ở dưới): đổi giá trị sang chữ, để in ra đúng như DuckDB hiển thị.

TIMESTAMP TZ (timestamp with time zone) là kiểu thời điểm của DuckDB dành cho giờ có múi giờ. Dù tên vậy, **nó không lưu múi giờ**: nó lưu số micro giây kể từ 0:00 UTC 1/1/1970. Khi **hiển thị** và **cắt theo ngày/giờ**, DuckDB dùng cài đặt `TimeZone`, mặc định là múi giờ của máy, nên hai máy có thể ra kết quả khác:

```

SET TimeZone = 'Asia/Ho_Chi_Minh';
SELECT date_trunc('day', TIMESTAMPTZ '2024-01-01 20:00:00+00')::VARCHAR; -- '2024-01-02
00:00:00+07'
SET TimeZone = 'UTC';
SELECT date_trunc('day', TIMESTAMPTZ '2024-01-01 20:00:00+00')::VARCHAR; -- '2024-01-01
00:00:00+00'

```

20:00 UTC ngày 1/1 đã là ngày 2/1 ở Việt Nam. Luôn viết `SET TimeZone = '...'` ở đầu script.

Với pandas 3, lấy cột `TIMESTAMPZ` về pandas cần cài gói múi giờ cũ `pytz`, nếu không sẽ báo "Required module 'pytz' failed to import".

12. Bẫy thường gặp

Bẫy	Triệu chứng	Sửa
Gán <code>df["a"][mask] = v</code> trên pandas 3	cột không đổi, chỉ có cảnh báo	<code>df.loc[mask, "a"] = v</code> (mục 9)
Bí danh cũ <code>H, M, T</code>	<code>ValueError</code> trên pandas 3	<code>h, ME, min</code> (mục 3)
Giả định mốc thời gian là nano giây	số nguyên nhỏ hơn 1.000 lần	<code>.dt.as_unit("ns")</code> trước (mục 9)
Đọc ngày kiểu Việt Nam không có <code>dayfirst=True</code>	05/01 thành 1 tháng 5	<code>dayfirst=True</code> hoặc <code>format=</code> (mục 3)
<code>resample(...).sum()</code> trên khoảng không có số liệu	ra 0 thay vì trống	<code>sum(min_count=1)</code> (mục 4)
<code>rolling(center=True)</code> hay <code>shift(-1)</code> làm feature	backtest (chấm mô hình trên quá khứ như đang dự báo thật) đẹp giả tạo	cửa sổ lùi + <code>shift(1)</code> (mục 5)
<code>rolling(k)</code> trên dữ liệu có chỗ hỏng	cửa sổ dài hơn dự định	<code>rolling("2h")</code> theo thời gian (mục 5)
<code>merge_asof</code> với bảng chưa xếp	<code>ValueError: ... keys must be sorted</code>	<code>sort_values</code> theo cột thời gian trước (mục 6)
Dữ liệu naive trộn nhiều múi giờ	sự kiện lệch vài giờ	đổi về UTC ngay khi đọc (mục 7)
Tự cộng 7 giờ cho Việt Nam	lệch giờ với dữ liệu trước 1975	<code>zoneinfo</code> / <code>tz_convert</code> (mục 7)
Giờ bị mất / lặp khi gần múi giờ Mỹ, châu Âu	<code>ValueError</code> ở ngày đổi giờ	<code>ambiguous=</code> , <code>nonexistent=</code> (mục 7)
<code>replace_time_zone</code> thay cho <code>convert_time_zone</code> (polars)	lệch giờ âm thầm	replace đổi nhãn, convert đổi giờ (mục 10)
DuckDB <code>date_trunc</code> trên <code>TIMESTAMPZ</code>	mỗi máy một kết quả	<code>SET TimeZone</code> đầu script (mục 11)

Nguồn

Tra cứu ngày 2026-09-17; code chạy lại ngày 2026-09-18.

- pandas 3.0.0 — What's new: <https://pandas.pydata.org/docs/whatsnew/v3.0.0.html>
- pandas `to_datetime`: https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html
- pandas `DataFrame.resample`: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.resample.html>
- pandas `DataFrame.rolling`: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.rolling.html>

- pandas `DataFrame.ewm`: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.ewm.html>
- pandas `merge_asof`: https://pandas.pydata.org/docs/reference/api/pandas.merge_asof.html
- pandas `Series.tz_localize`: https://pandas.pydata.org/docs/reference/api/pandas.Series.tz_localize.html
- NumPy `quantile`: <https://numpy.org/doc/stable/reference/generated/numpy.quantile.html>
- NumPy random `Generator`: <https://numpy.org/doc/stable/reference/random/generator.html>
- Hyndman, R.J. & Fan, Y. (1996). Sample quantiles in statistical packages. *The American Statistician* 50(4), 361–365.
- Python `zoneinfo`: <https://docs.python.org/3/library/zoneinfo.html>
- IANA tz database, tệp `asia`: <https://data.iana.org/time-zones/tzdb/asia>
- polars `join_asof`: https://docs.pola.rs/api/python/stable/reference/dataframe/api/polars.DataFrame.join_asof.html
- polars `group_by_dynamic`, `dt.replace_time_zone`, `dt.convert_time_zone`: <https://docs.pola.rs/api/python/stable/reference/>
- DuckDB ASOF JOIN: https://duckdb.org/docs/current/guides/sql_features/asof_join.html
- DuckDB hàm thời gian: <https://duckdb.org/docs/current/sql/functions/timestamp.html>
- DuckDB TIMESTAMP / TIMESTAMPTZ: https://duckdb.org/docs/current/sql/data_types/timestamp.html